Table 7.2. Computing an expected value from a probability distribution

Math	Python

$\mathbb{E}[P] = \sum x \cdot P(x)$	`>>> x = np.array([1,2,3,4,5,6])`
	`>>> p = np.array([0.1,0.1,0.1,0.1,0.2,0.4])`
	`>>> def expected_value(x,p):`
	`>>>  return x @ p`
	`>>> expected_value(x,p)`
	`4.4`

The expected value operator (where *operator* is another term for *function*) is denoted $\mathbb{E}$, and it's a function that takes in a probability distribution and returns its expected value. It works by taking a value, x, multiplying by its associated probability, $P(x)$, and summing for all possible values of x.

In Python, if $P(x)$ is represented as a numpy array of probabilities, `probs`, and another numpy array of `outcomes` (the sample space), the expected value is

```
>>> import numpy as np
>>> probs = np.array([0.6, 0.1, 0.1, 0.1, 0.1])
>>> outcomes = np.array([18, 21, 17, 17, 21])
>>> expected_value = 0.0
>>> for i in range(probs.shape[0]):
>>>         expected_value += probs[i] * outcomes[i]

>>> expected_value
18.4
```

Alternatively, the expected value can be computed as the inner (dot) product between the `probs` array and the `outcomes` array, since the inner product does the same thing—it multiplies each corresponding element in the two arrays and sums them all.

```
>>> expected_value = probs @ outcomes
>>> expected_value
18.4
```

A discrete probability distribution means that its sample space is a finite set, or in other words, only a finite number of possible outcomes can occur. A coin toss, for example, can only have one of two outcomes.